RAG

What is RAG?

Up to this point, we have seen two ways to adapt an LLM to a task:

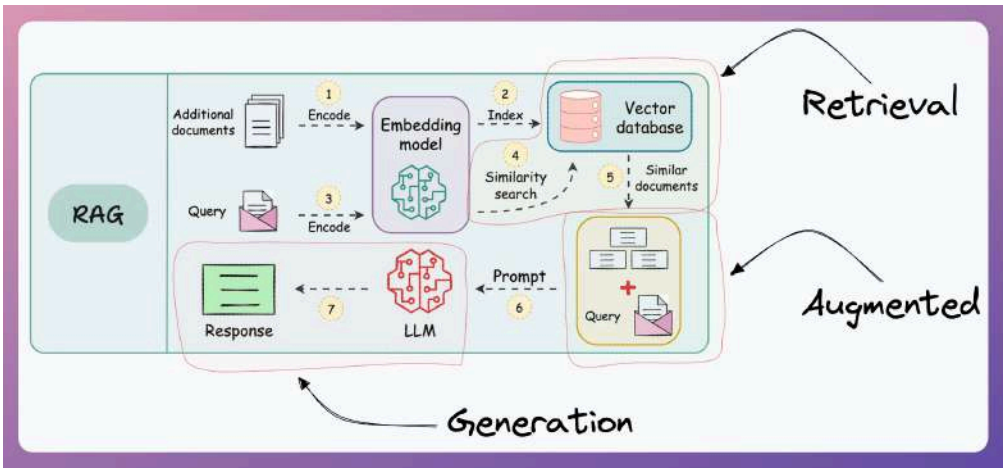
- Prompt Engineering - which steers the model at inference time
- Fine-tuning - which adjusts its internal parameters.

Both approaches are powerful, but they share one fundamental limitation: the model can only use the knowledge it already contains.

LLMs do not automatically know new information, private data, company documents, or anything that appeared after their training cutoff.

Retraining them repeatedly to stay updated is impractical and expensive.

This is where Retrieval-Augmented Generation (RAG) comes in. Let's break it down:



- Retrieval: Accessing and retrieving information from a knowledge source, such as a database or memory.
- Augmented: Enhancing or enriching something, in this case, the text generation process, with additional information or context.
- Generation: The process of creating or producing something, in this context, generating text or language.

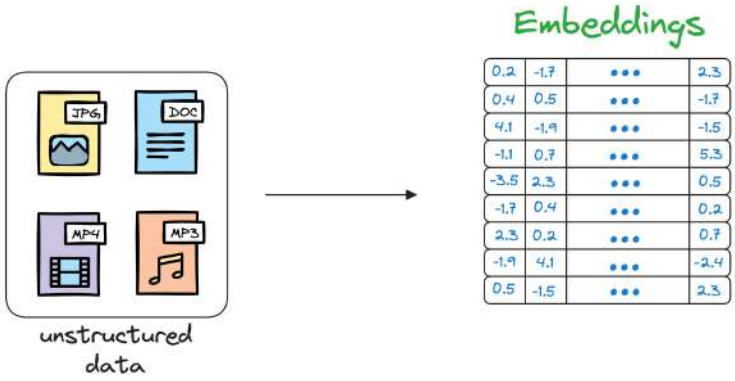
With RAG, the language model can use the retrieved information (which is expected to be reliable) from the vector database to ensure that its responses are grounded in real-world knowledge and context, reducing the likelihood of hallucinations.

This makes the model's responses more accurate, reliable, and contextually relevant, while also ensuring that we don't have to train the LLM repeatedly on new data. This makes the model more "real-time" in its responses.

To understand how RAG actually works in practice, we first need to understand vector databases - the storage layer that powers retrieval.

What are vector databases?

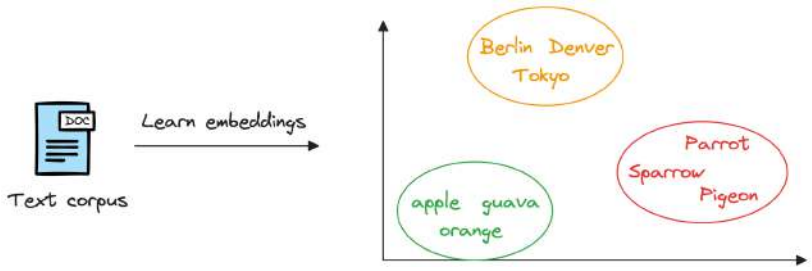
Simply put, a vector database stores unstructured data (text, images, audio, video, etc.) in the form of vector embeddings.



Each data point, whether a word, a document, an image, or any other entity, is transformed into a numerical vector using ML techniques (which we shall see ahead).

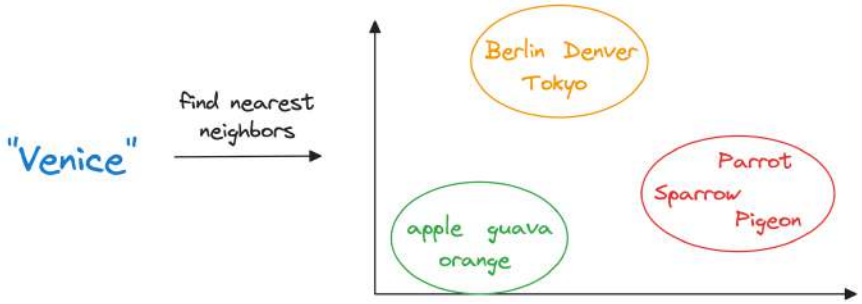
This numerical vector is called an embedding, and the model is trained in such a way that these vectors capture the essential features and characteristics of the underlying data.

Considering word embeddings, for instance, we may discover that in the embedding space, the embeddings of fruits are found close to each other, which cities form another cluster, and so on.



This shows that embeddings can learn the semantic characteristics of entities they represent (provided they are trained appropriately).

Once stored in a vector database, we can retrieve original objects that are similar to the query we wish to run on our unstructured data.



In other words, encoding unstructured data allows us to run many sophisticated operations like similarity search, clustering, and classification over it, which otherwise is difficult with traditional databases.

To exemplify, when an e-commerce website provides recommendations for similar items or searches for a product based on the input query, we're (in most cases) interacting with vector databases behind the scenes.

The purpose of vector databases in RAG

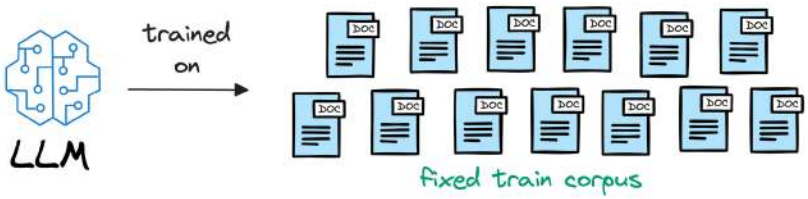
At this point, one interesting thing to learn is how exactly LLMs take advantage of vector databases.

The biggest confusion that people typically face is:

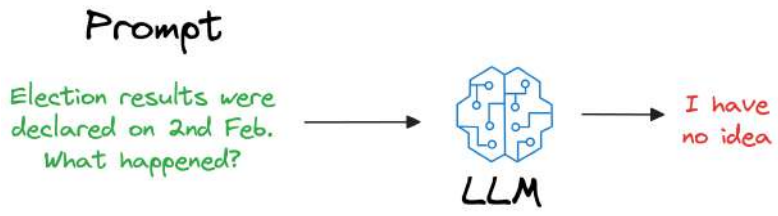
Once we have trained our LLM, it will have some model weights for text generation. Where do vector databases fit in here?

Let's understand this.

To begin, we must understand that an LLM is deployed after learning from a static version of the corpus it was fed during training.



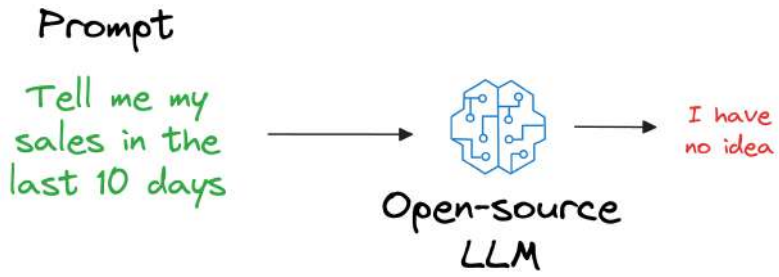
For instance, if the model was deployed after considering the data until 31st Jan 2024, and we use it, say, a week after training, it will have no clue about what happened in those days.



Repeatedly training a new model (or adapting the latest version) every single day on new data is impractical and cost-ineffective. In fact, LLMs can take weeks to train.

Also, what if we open-sourced the LLM and someone else wants to use it on their privately held dataset, which, of course, was not shown during training?

As expected, the LLM will have no clue about it.

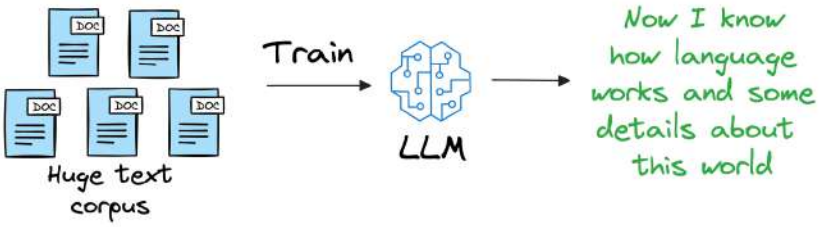


But if you think about it, is it really our objective to train an LLM to know every single thing in the world?

Not at all!

That's not our objective.

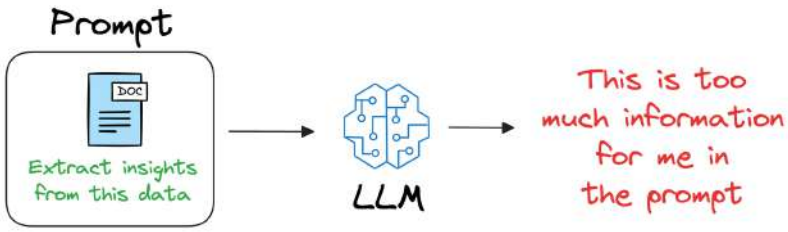
Instead, it is more about helping the LLM learn the overall structure of the language, and how to understand and generate it.



So, once we have trained this model on a ridiculously large enough training corpus, it can be expected that the model will have a decent level of language understanding and generation capabilities.

Thus, if we could figure out a way for LLMs to look up new information they were not trained on and use it in text generation (without training the model again), that would be great!

One way could be to provide that information in the prompt itself.

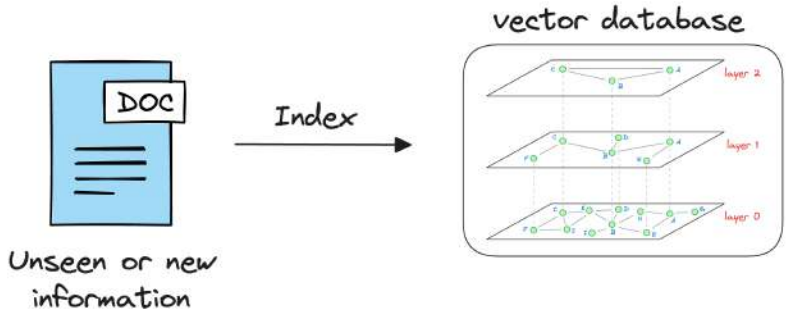


But since LLMs usually have a limit on the context window (number of words/tokens they can accept), the additional information can exceed that limit.

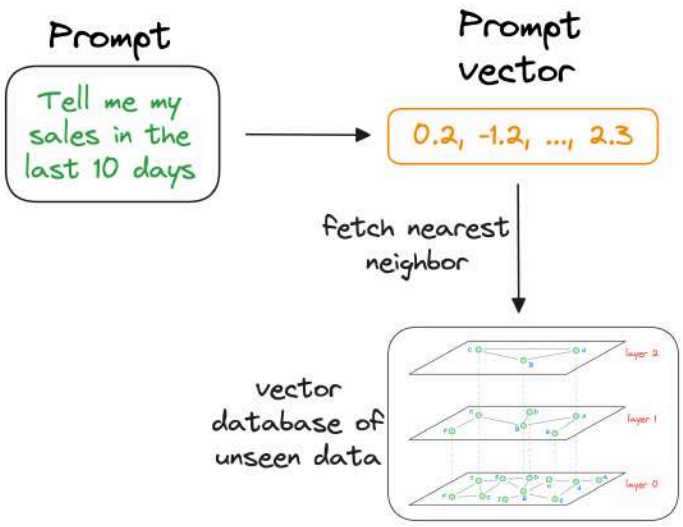
Vector databases solve this problem.

As discussed earlier, vector databases store information in the form of vectors, where each vector captures semantic information about the piece of text being encoded.

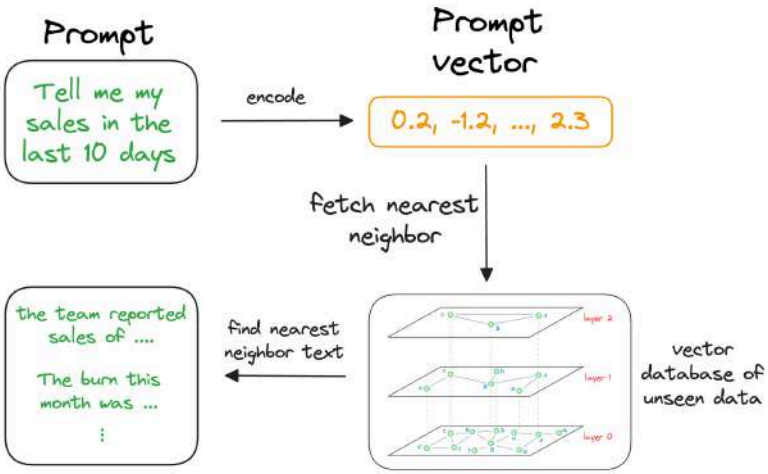
Thus, we can maintain our available information in a vector database by encoding it into vectors using an embedding model.



When the LLM needs to access this information, it can query the vector database using an approximate similarity search with the prompt vector to find content that is similar to the input query vector.

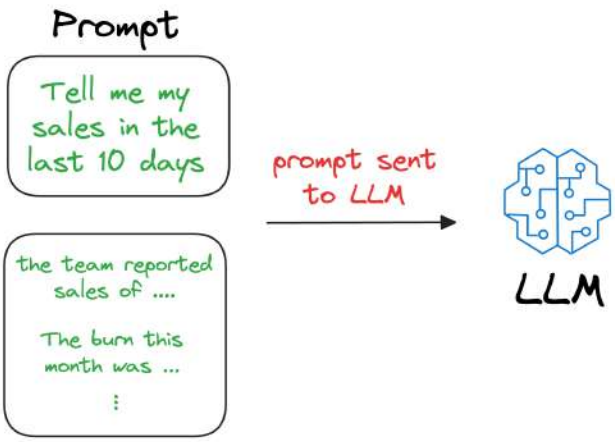


Once the approximate nearest neighbors have been retrieved, we gather the context corresponding to those specific vectors, which were stored at the time of indexing the data in the vector database (this raw data is stored as payload, which we will learn during implementation).



The above search process retrieves context that is similar to the query vector, which represents the context or topic the LLM is interested in.

We can augment this retrieved content along with the actual prompt provided by the user and give it as input to the LLM.



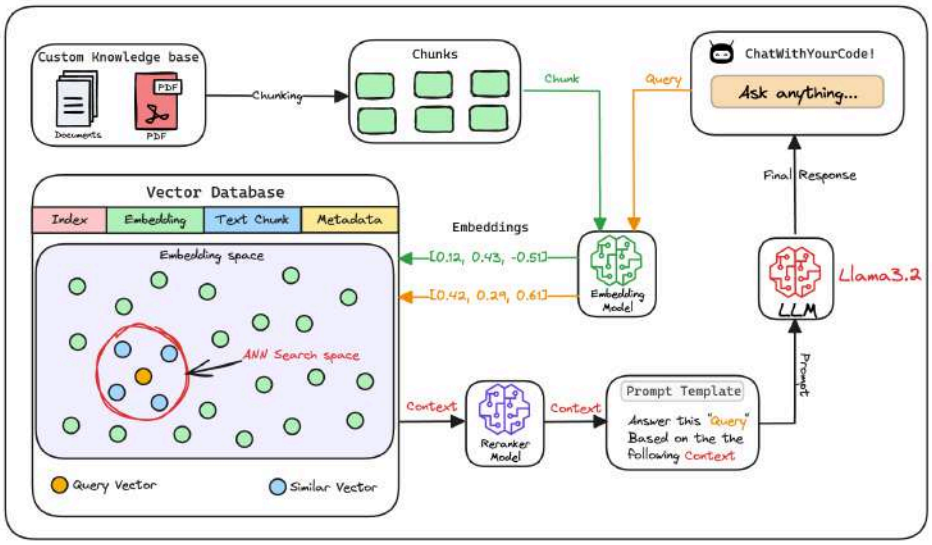
Consequently, the LLM can easily incorporate this info while generating text because it now has the relevant details available in the prompt.

Now that we understand the purpose, let's get into the technical details.

Workflow of a RAG system

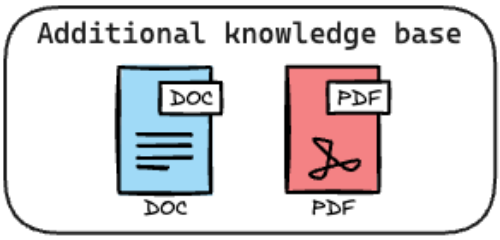
To build a RAG system, it's crucial to understand the foundational components that go into it and how they interact. Thus, in this section, let's explore each element in detail.

Here's an architecture diagram of a typical RAG setup:



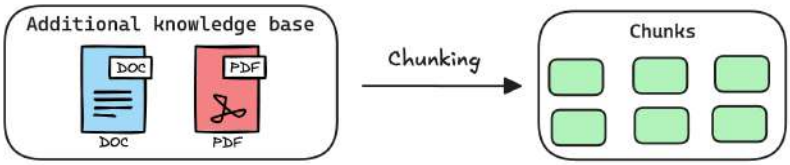
Let's break it down step by step.

We start with some external knowledge that wasn't seen during training, and we want to augment the LLM with:

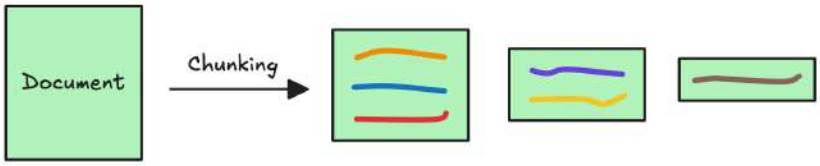


#1) Create chunks

The first step is to break down this additional knowledge into chunks before embedding and storing it in the vector database.



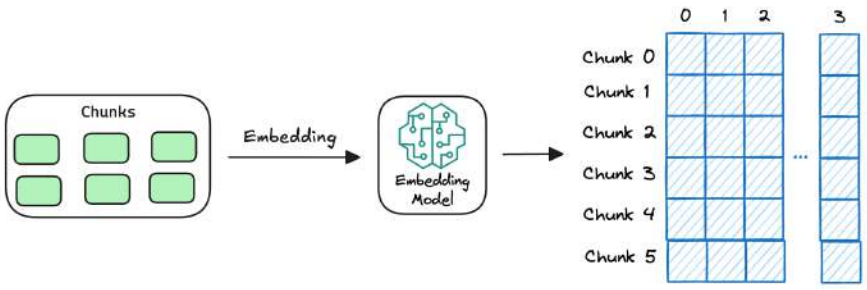
We do this because the additional document(s) can be pretty large. Thus, it is important to ensure that the text fits the input size of the embedding model.



Moreover, if we don't chunk, the entire document will have a single embedding, which won't be of any practical use to retrieve relevant context.

#2) Generate embeddings

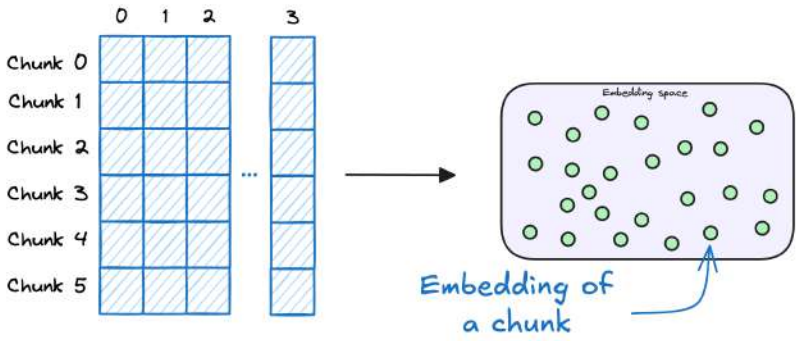
After chunking, we embed the chunks using an embedding model.



Since these are "context embedding models" (not word embedding models), models like bi-encoders are highly relevant here.

#3) Store embeddings in a vector database

These embeddings are then stored in the vector database:



This shows that a vector database acts as a memory for your RAG application since this is precisely where we store all the additional knowledge, using which, the user's query will be answered.

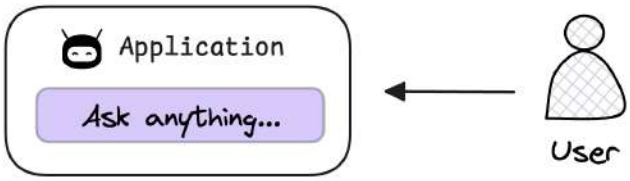
A vector database also stores the metadata and original content along with the vector embeddings.

With that, our vector database has been created and information has been added. More information can be added to this if needed.

Now, we move to the query step.

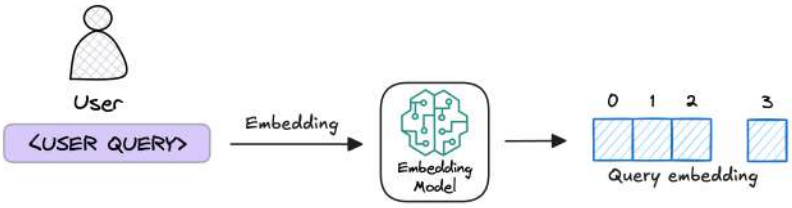
#4) User input query

Next, the user inputs a query, a string representing the information they're seeking.



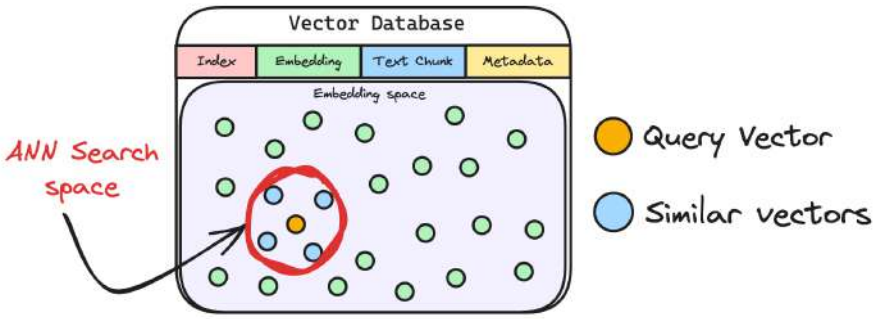
#5) Embed the query

This query is transformed into a vector using the same embedding model we used to embed the chunks earlier in Step 2.

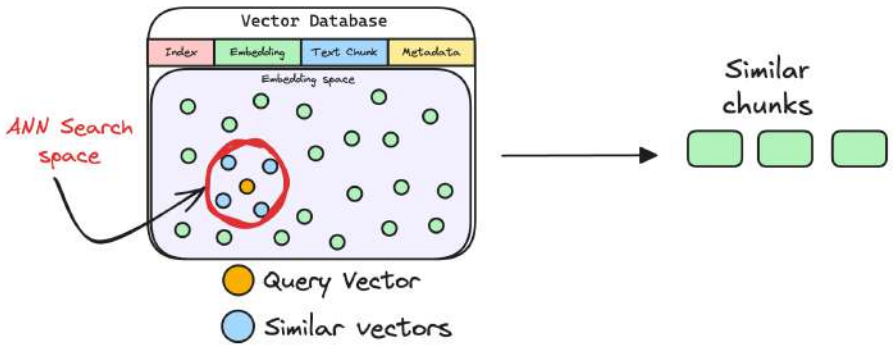


#6) Retrieve similar chunks

The vectorized query is then compared against our existing vectors in the database to find the most similar information.



The vector database returns the k (a pre-defined parameter) most similar documents/chunks (using approximate nearest neighbor search).

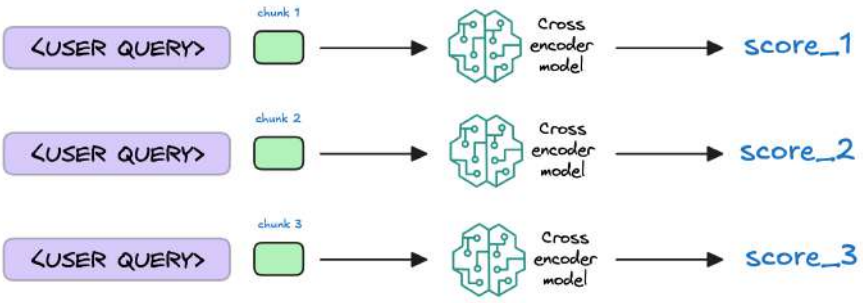


It is expected that these retrieved documents contain information related to the query, providing a basis for the final response generation.

#7) Re-rank the chunks

After retrieval, the selected chunks might need further refinement to ensure the most relevant information is prioritized.

In this re-ranking step, a more sophisticated model (often a cross-encoder) evaluates the initial list of retrieved chunks alongside the query to assign a relevance score to each chunk.



This process rearranges the chunks so that the most relevant ones are prioritized for the response generation.

That said, not every RAG app implements this, and typically, they just rely on the similarity scores obtained in step 6 while retrieving the relevant context from the vector database.

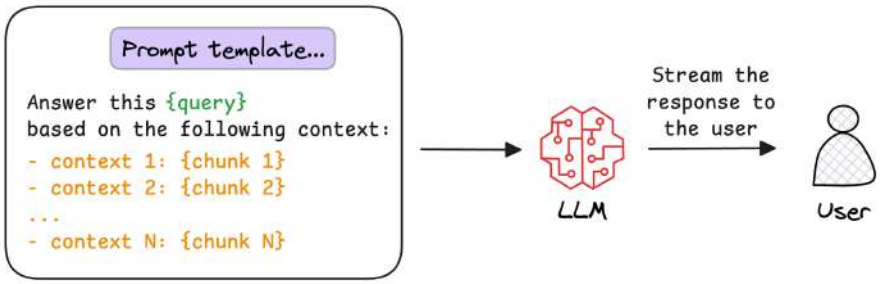
#8) Generate the final response

Almost done!

Once the most relevant chunks are re-ranked, they are fed into the LLM.

This model combines the user's original query with the retrieved chunks in a prompt template to generate a response that synthesizes information from the selected documents.

This is depicted below:

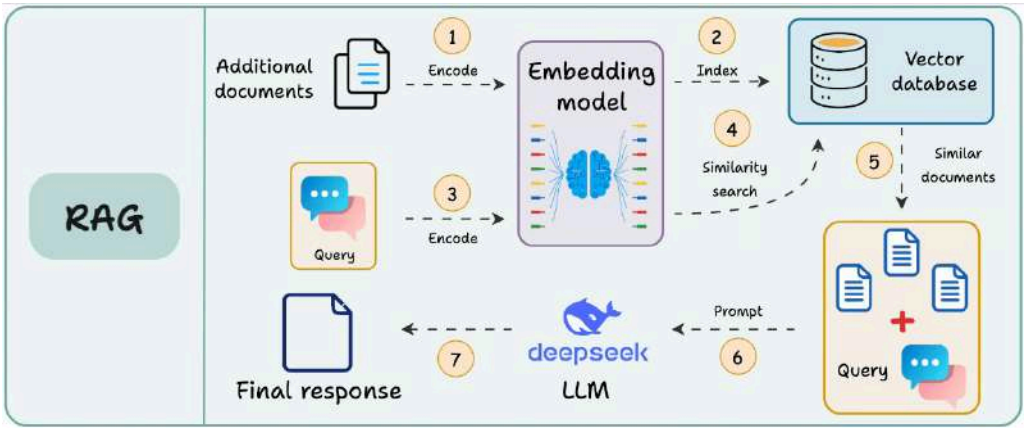


The LLM leverages the context provided by the chunks to generate a coherent and contextually relevant answer that directly addresses the user's query.

Since chunking is the very first step in any RAG pipeline, it's important to understand the different ways it can be done.

5 chunking strategies for RAG

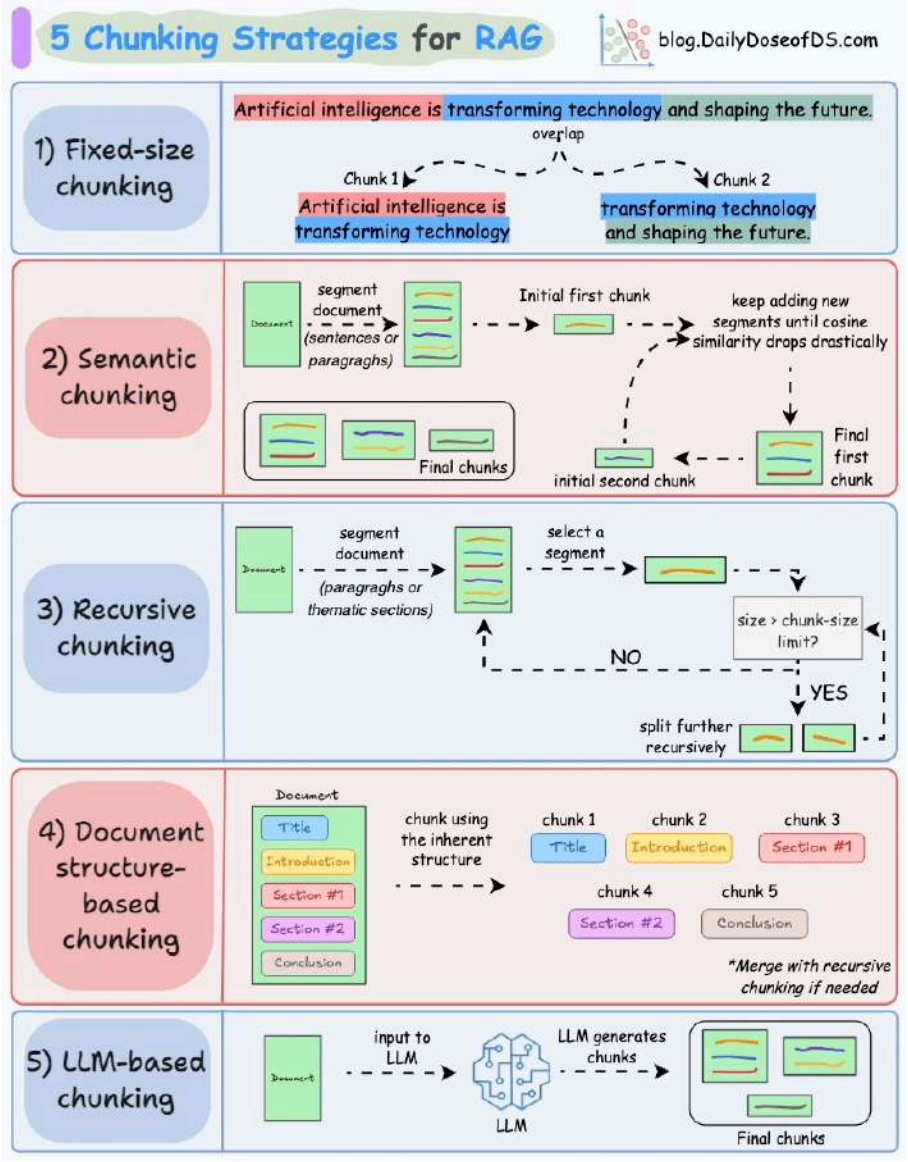
Here's the typical workflow of RAG:



Since the additional document(s) can be large, step 1 also involves chunking, wherein a large document is divided into smaller/manageable pieces.

This step is crucial since it ensures the text fits the input size of the embedding model.

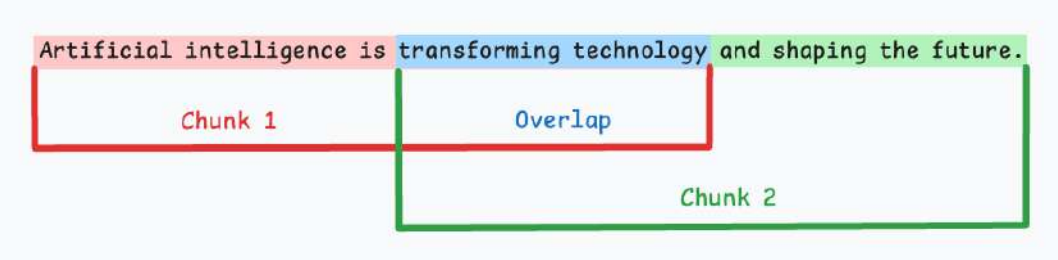
Here are five chunking strategies for RAG:



Let's understand them!

1) Fixed-size chunking

Split the text into uniform segments based on a pre-defined number of characters, words, or tokens.

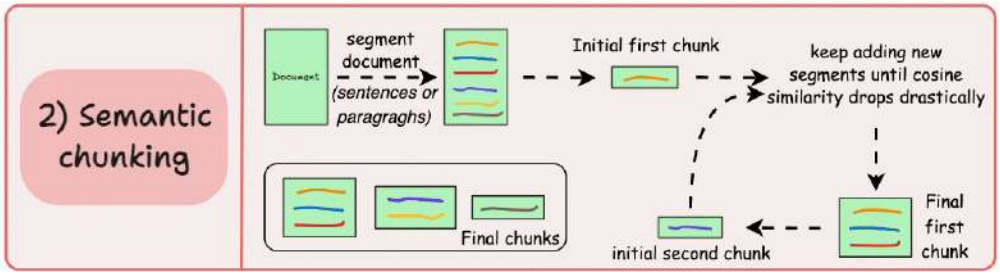


Since a direct split can disrupt the semantic flow, it is recommended to maintain some overlap between two consecutive chunks (the blue part above).

This is simple to implement. Also, since all chunks are of equal size, it simplifies batch processing.

But this usually breaks sentences (or ideas) in between. Thus, important information will likely get distributed between chunks.

2) Semantic chunking



Segment the document based on meaningful units like sentences, paragraphs, or thematic sections.

Next, create embeddings for each segment.

Let's say we start with the first segment and its embedding.

If the first segment's embedding has a high cosine similarity with that of the second segment, both segments form a chunk.

This continues until cosine similarity drops significantly.

The moment it does, we start a new chunk and repeat.

Here's what the output could look like:

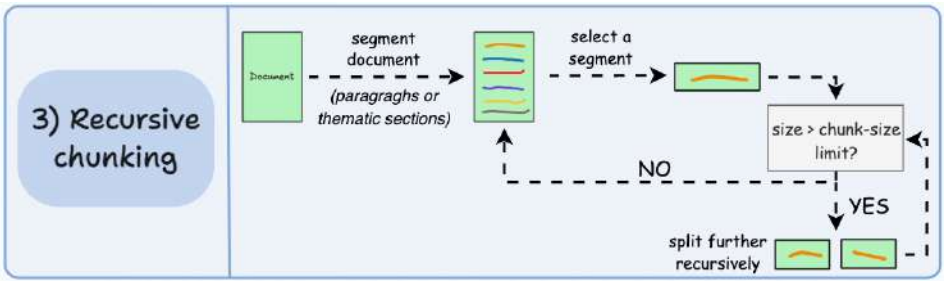
```
Artificial intelligence is transforming industries by automating
processes, enhancing decision-making, and providing insights through
data analysis. Machine learning, a subset of AI, enables systems to learn
and improve from experience without explicit programming. Deep learning,
a branch of machine learning, uses neural networks with multiple layers
to model complex patterns in data.
```

Unlike fixed-size chunks, this maintains the natural flow of language and preserves complete ideas.

Since each chunk is richer, it improves the retrieval accuracy, which, in turn, produces more coherent and relevant responses by the LLM.

A minor problem is that it depends on a threshold to determine if cosine similarity has dropped significantly, which can vary from document to document.

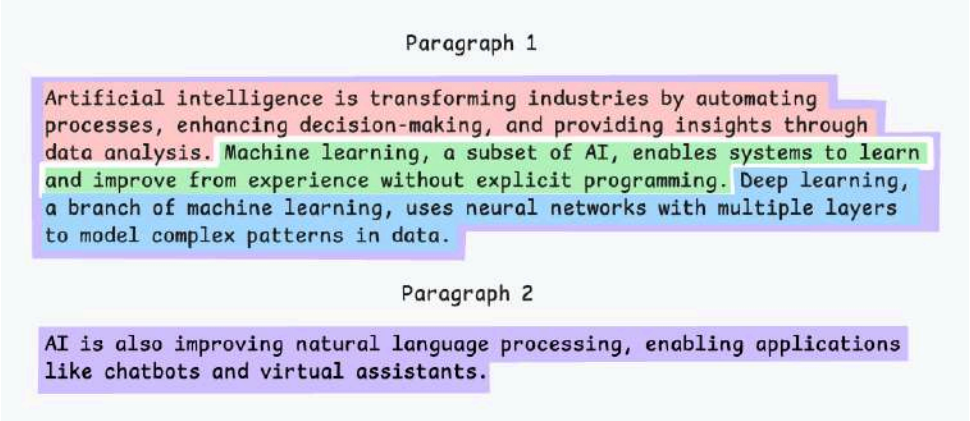
3) Recursive chunking



First, chunk based on inherent separators like paragraphs, or sections.

Next, split each chunk into smaller chunks if the size exceeds a pre-defined chunk size limit. If, however, the chunk fits the chunk-size limit, no further splitting is done.

Here's what the output could look like:



As shown above:

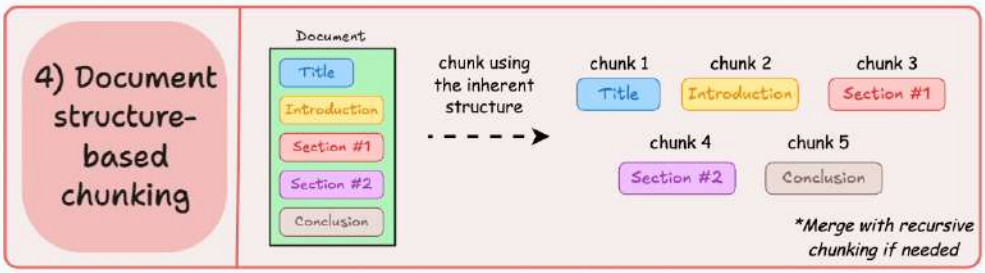
First, we define two chunks (the two paragraphs in purple).

Next, paragraph 1 is further split into smaller chunks.

Unlike fixed-size chunks, this approach also maintains the natural flow of language and preserves complete ideas.

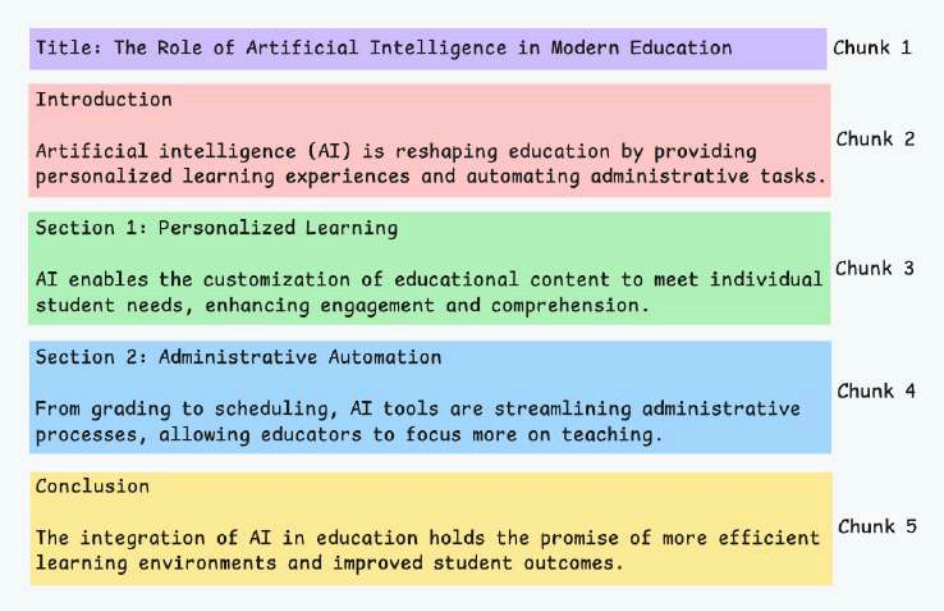
However, there is some extra overhead in terms of implementation and computational complexity.

4) Document structure-based chunking



It utilizes the inherent structure of documents, like headings, sections, or paragraphs, to define chunk boundaries. This way, it maintains structural integrity by aligning with the document's logical sections.

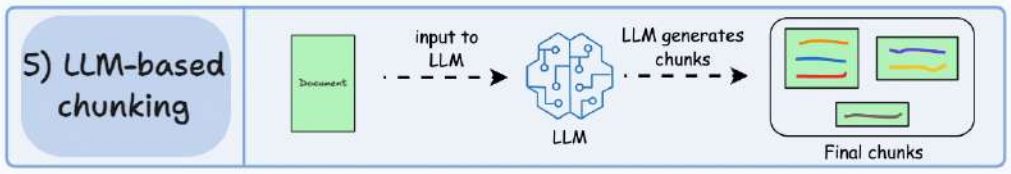
Here's what the output could look like:



That said, this approach assumes that the document has a clear structure, which may not be true.

Also, chunks may vary in length, possibly exceeding model token limits. You can try merging it with recursive splitting.

5) LLM-based chunking



Prompt the LLM to generate semantically isolated and meaningful chunks.

This method ensures high semantic accuracy since the LLM can understand context and meaning beyond simple heuristics (used in the above four approaches).

But this is the most computationally demanding chunking technique of all five techniques discussed here.

Also, since LLMs typically have a limited context window, that is something to be taken care of.

Each technique has its own advantages and trade-offs.

We have observed that semantic chunking works pretty well in many cases, but again, you need to test.

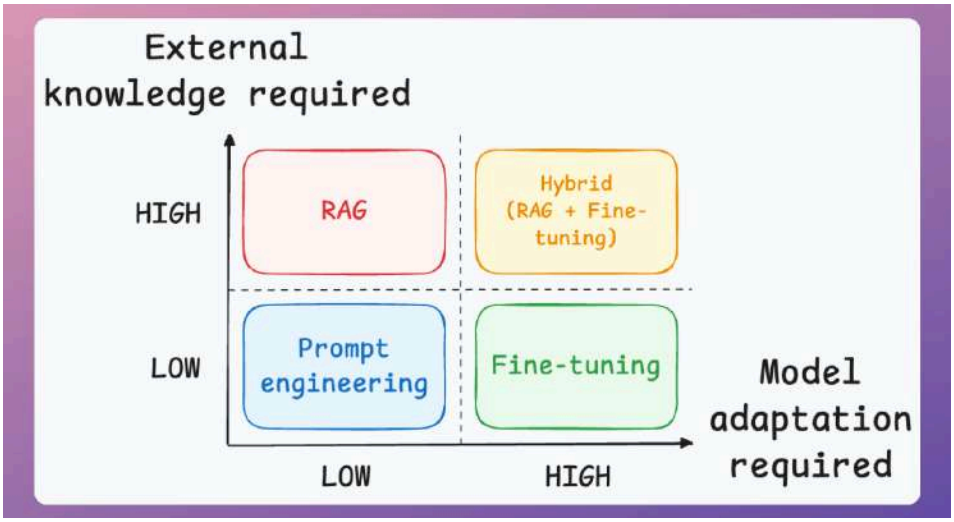
The choice will depend on the nature of your content, the capabilities of the embedding model, computational resources, etc.

Prompting vs. RAG vs. Finetuning?

If you are building real-world LLM-based apps, it is unlikely you can start using the model right away without adjustments. To maintain high utility, you either need:

- Prompt engineering
- Fine-tuning
- RAG
- Or a hybrid approach (RAG + fine-tuning)

The following visual will help you decide which one is best for you:



Two important parameters guide this decision:

- The amount of external knowledge required for your task.
- The amount of adaptation you need. Adaptation, in this case, means changing the behavior of the model, its vocabulary, writing style, etc.

For instance, an LLM might find it challenging to summarize the transcripts of company meetings because speakers might be using some internal vocabulary in their discussions.

So here's the simple takeaway:

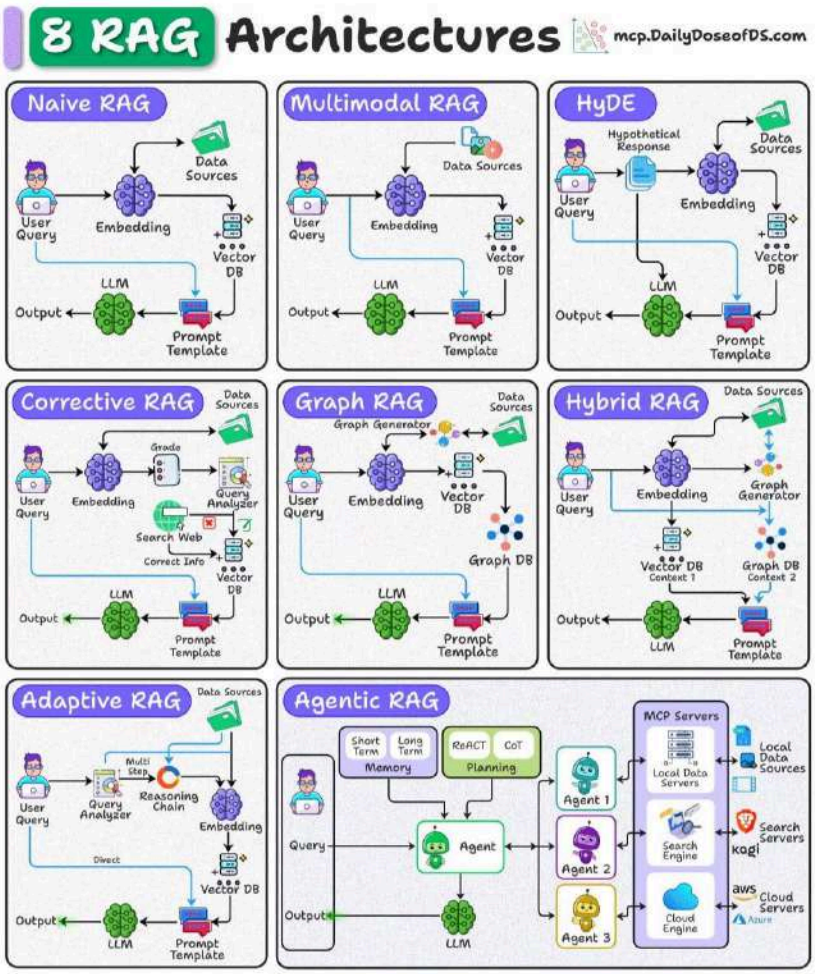
- Use RAGs to generate outputs based on a custom knowledge base if the vocabulary & writing style of the LLM remains the same.
- Use fine-tuning to change the structure (behaviour) of the model than knowledge.
- Prompt engineering is sufficient if you don't have a custom knowledge base and don't want to change the behavior.
- And finally, if your application demands a custom knowledge base and a change in the model's behavior, use a hybrid (RAG + Fine-tuning) approach.

That's it!

Once you've decided that RAG is the right approach, the next step is choosing the right RAG architecture for your use case.

8 RAG architectures

We prepared the following visual that details 8 types of RAG architectures used in AI systems:



Let's discuss them briefly:

1) Naive RAG

Retrieves documents purely based on vector similarity between the query embedding and stored embeddings.

Works best for simple, fact-based queries where direct semantic matching suffices.

2) Multimodal RAG

Handles multiple data types (text, images, audio, etc.) by embedding and retrieving across modalities.

Ideal for cross-modal retrieval tasks like answering a text query with both text and image context.

3) HyDE

Queries are not semantically similar to documents.

This technique generates a hypothetical answer document from the query before retrieval.

Uses this generated document's embedding to find more relevant real documents.

4) Corrective RAG

Validates retrieved results by comparing them against trusted sources (e.g., web search).

Ensures up-to-date and accurate information, filtering or correcting retrieved content before passing to the LLM.

5) Graph RAG

Converts retrieved content into a knowledge graph to capture relationships and entities.

Enhances reasoning by providing structured context alongside raw text to the LLM.

6) Hybrid RAG

Combines dense vector retrieval with graph-based retrieval in a single pipeline.

Useful when the task requires both unstructured text and structured relational data for richer answers.

7) Adaptive RAG

Dynamically decides if a query requires a simple direct retrieval or a multi-step reasoning chain.

Breaks complex queries into smaller sub-queries for better coverage and accuracy.

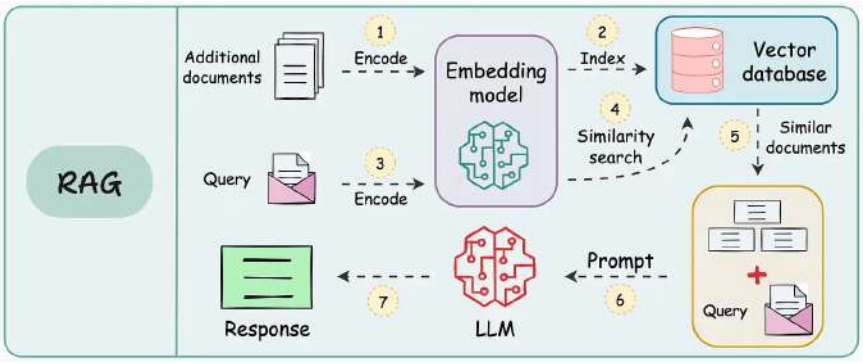
8) Agentic RAG

Uses AI agents with planning, reasoning (ReAct, CoT), and memory to orchestrate retrieval from multiple sources.

Best suited for complex workflows that require tool use, external APIs, or combining multiple RAG techniques.

RAG vs Agentic RAG

These are some issues with the traditional RAG system:



These systems retrieve once and generate once. This means if the retrieved context isn't enough, the LLM can not dynamically search for more information.

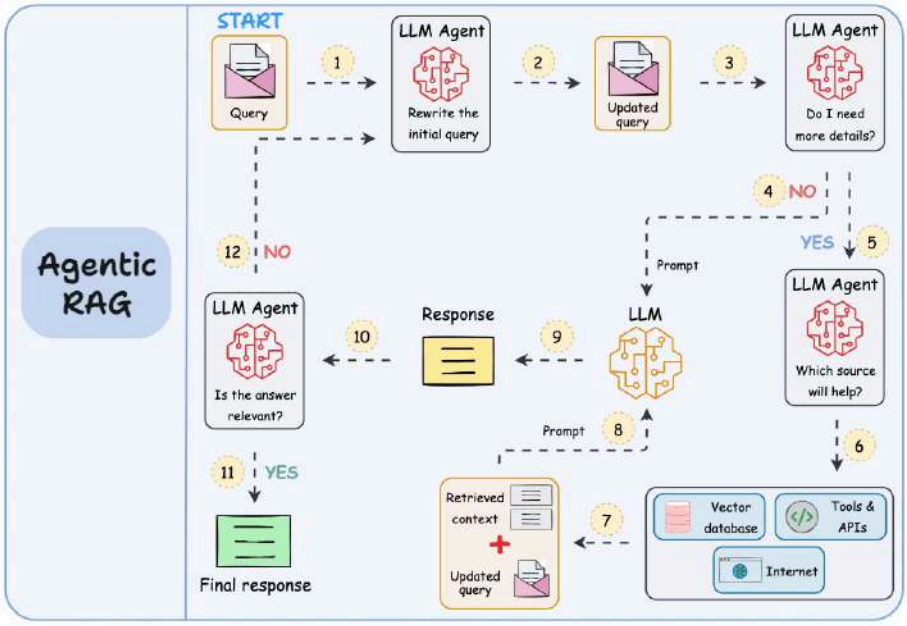
RAG systems may provide relevant context but don't reason through complex queries. If a query requires multiple retrieval steps, traditional RAG falls short.

There's little adaptability. The LLM can't modify its strategy based on the problem at hand.

Due to this, Agentic RAG is becoming increasingly popular. Let's understand this in more detail.

Agentic RAG

The workflow of agentic RAG is depicted below:



Note: The diagram above is one of many blueprints that an agentic RAG system may possess. You can adapt it according to your specific use case.

As shown above, the idea is to introduce agentic behaviors at each stage of RAG.

Think of agents as someone who can actively think through a task - planning, adapting, and iterating until they arrive at the best solution, rather than just

following a defined set of instructions. The powerful capabilities of LLMs make this possible.

Let's understand this step-by-step:

Steps 1-2) The user inputs the query, and an agent rewrites it (removing spelling mistakes, simplifying it for embedding, etc.)

Step 3) Another agent decides whether it needs more details to answer the query.

Step 4) If not, the rewritten query is sent to the LLM as a prompt.

Step 5-8) If yes, another agent looks through the relevant sources it has access to (vector database, tools & APIs, and the internet) and decides which source should be useful. The relevant context is retrieved and sent to the LLM as a prompt.

Step 9) Either of the above two paths produces a response.

Step 10) A final agent checks if the answer is relevant to the query and context.

Step 11) If yes, return the response.

Step 12) If not, go back to Step 1. This procedure continues for a few iterations until the system admits it cannot answer the query.

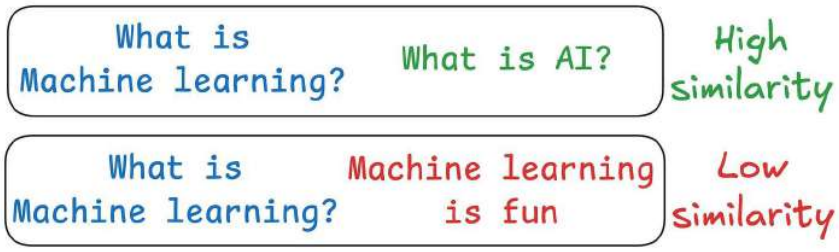
This makes the RAG much more robust since, at every step, agentic behavior ensures that individual outcomes are aligned with the final goal.

That said, it is also important to note that building RAG systems typically boils down to design preferences/choices.

Apart from agentic approaches, another important improvement over traditional RAG comes from better retrieval itself - one popular method being HyDE.

Traditional RAG vs HyDE

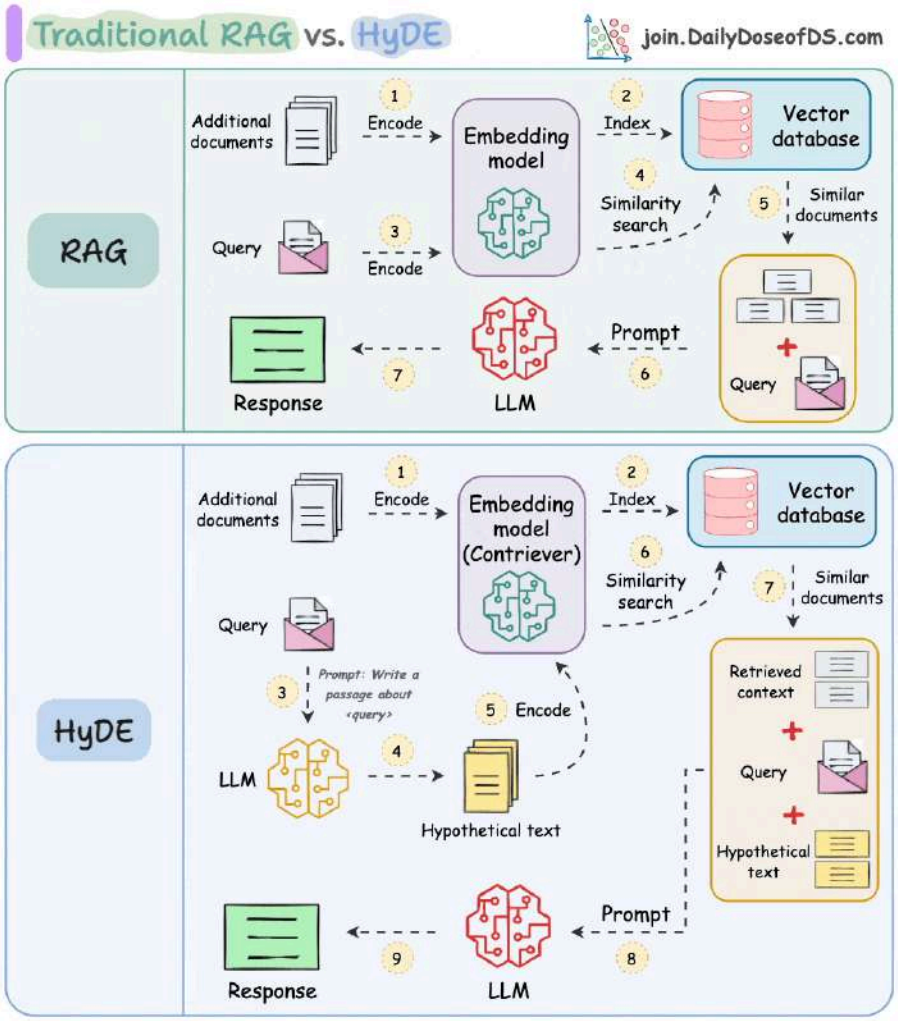
Another critical problem with the traditional RAG system is that questions are not semantically similar to their answers.



As a result, several irrelevant contexts get retrieved during the retrieval step due to a higher cosine similarity than the documents actually containing the answer.

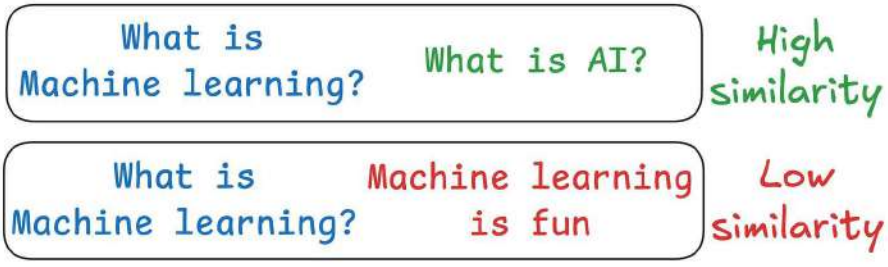
HyDE solves this.

The following visual depicts how it differs from traditional RAG and HyDE.



Let's understand this in more detail.

As mentioned earlier, questions are not semantically similar to their answers, which leads to several irrelevant contexts during retrieval.



HyDE handles this as follows:

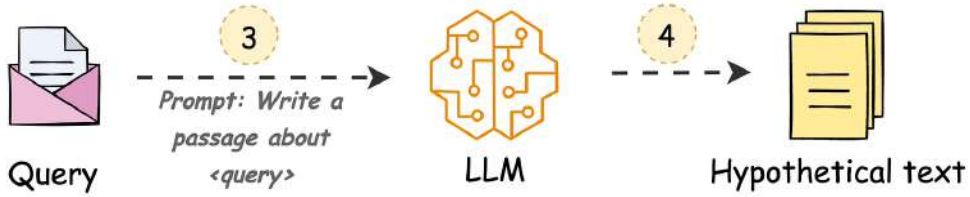
Use an LLM to generate a hypothetical answer H for the query Q (this answer does not have to be entirely correct).

Embed the answer using a contriever model to get E (Bi-encoders are famously used here).

Use the embedding E to query the vector database and fetch relevant context (C).

Pass the hypothetical answer H + retrieved-context C + query Q to the LLM to produce an answer.

Done!



Now, of course, the hypothetical generated will likely contain hallucinated details.

But this does not severely affect the performance due to the contriever model - one which embeds.

More specifically, this model is trained using contrastive learning and it also functions as a near-lossless compressor whose task is to filter out the hallucinated details of the fake document.

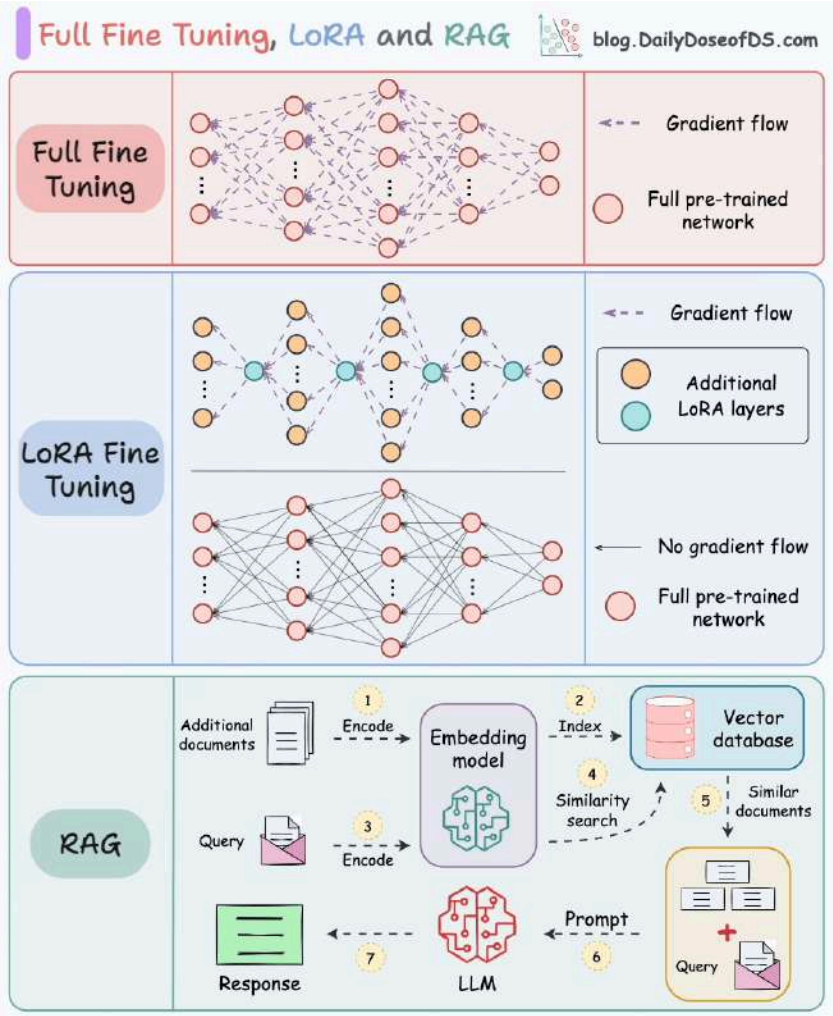
This produces a vector embedding that is expected to be more similar to the embeddings of actual documents than the question is to the real documents:



Several studies have shown that HyDE improves the retrieval performance compared to the traditional embedding model.

But this comes at the cost of increased latency and more LLM usage.

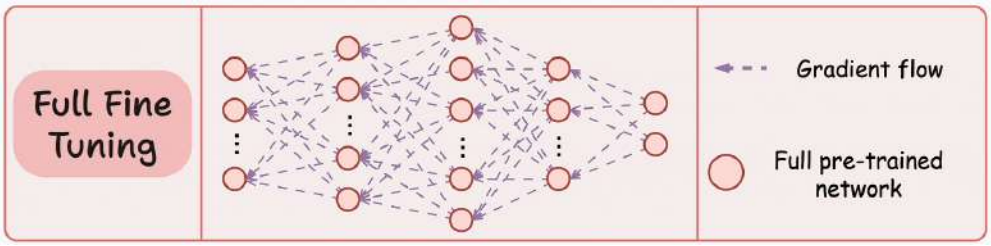
Full-model Fine-tuning vs. LoRA vs. RAG



All three techniques are used to augment the knowledge of an existing model with additional data.

1) Full fine-tuning

Fine-tuning means adjusting the weights of a pre-trained model on a new dataset for better performance.



While this fine-tuning technique has been successfully used for a long time, problems arise when we use it on much larger models — LLMs, for instance, primarily because of:

Their size.

The cost involved in fine-tuning all weights.

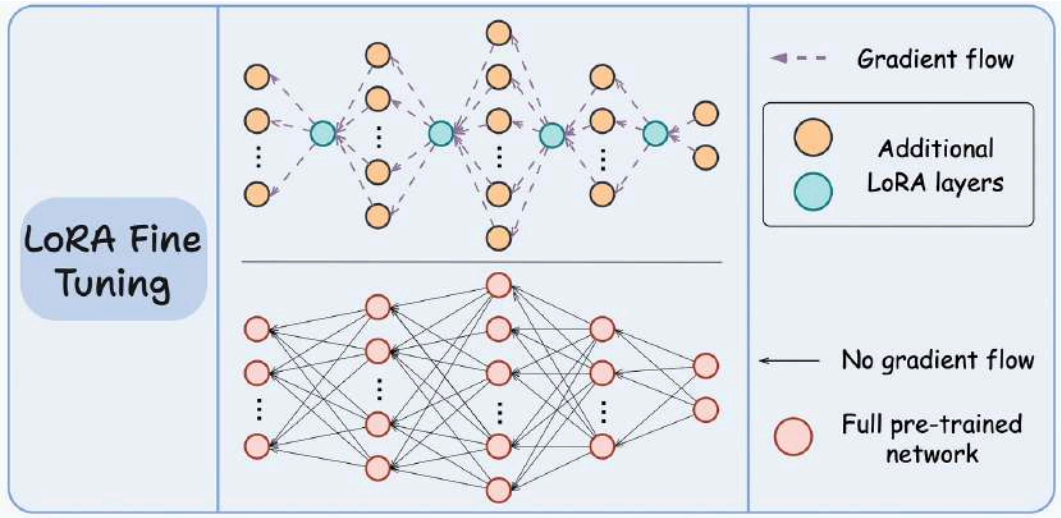
The cost involved in maintaining all large fine-tuned models.

2) LoRA fine-tuning

LoRA fine-tuning addresses the limitations of traditional fine-tuning.

The core idea is to decompose the weight matrices (some or all) of the original model into low-rank matrices and train them instead.

For instance, in the graphic below, the bottom network represents the large pre-trained model, and the top network represents the model with LoRA layers.



The idea is to train only the LoRA network and freeze the large model.

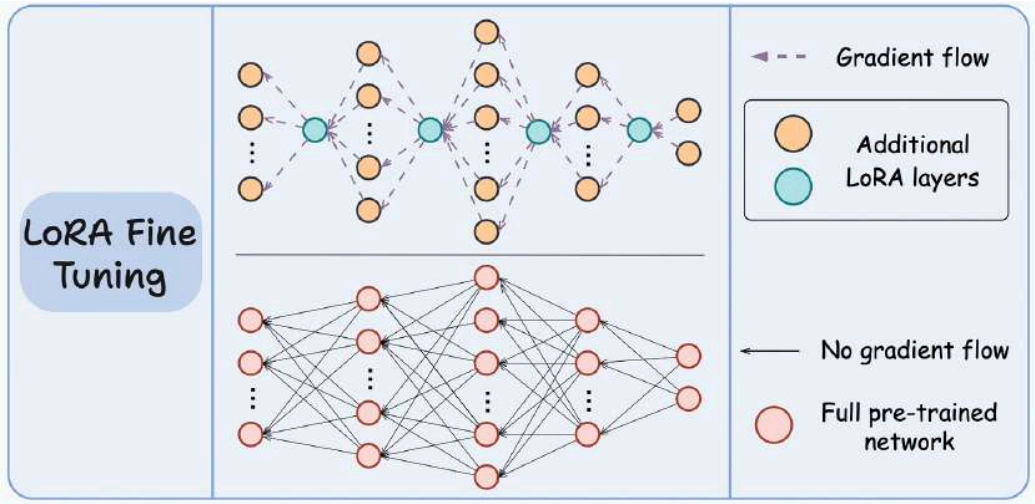
Looking at the above visual, you might think:

But the LoRA model has more neurons than the original model. How does that help?

To understand this, you must make it clear that neurons don't have anything to do with the memory of the network. They are just used to illustrate the dimensionality transformation from one layer to another.

It is the weight matrices (or the connections between two layers) that take up memory.

Thus, we must be comparing these connections instead:

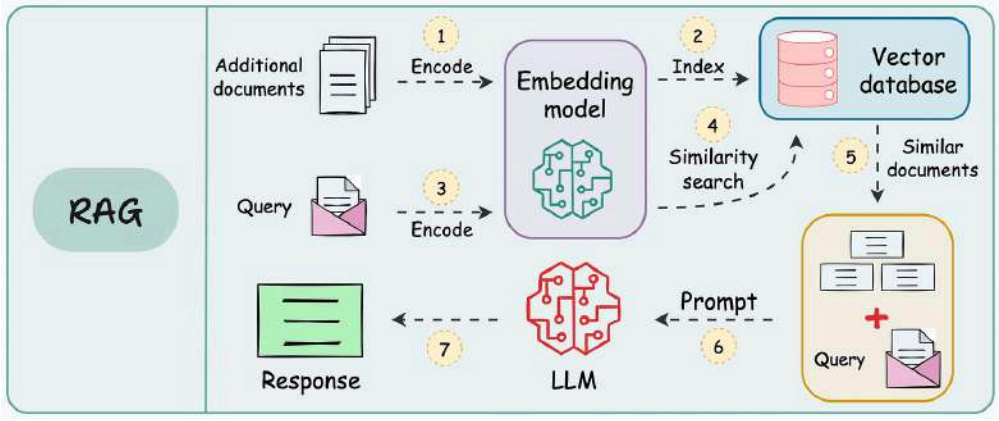


Looking at the above visual, it is pretty clear that the LoRA network has relatively very few connections.

3) RAG

Retrieval augmented generation (RAG) is another pretty cool way to augment neural networks with additional information, without having to fine-tune the model.

This is illustrated below:



There are 7 steps, which are also marked in the above visual:

Step 1-2: Take additional data, and dump it in a vector database after embedding. (This is only done once. If the data is evolving, just keep dumping the

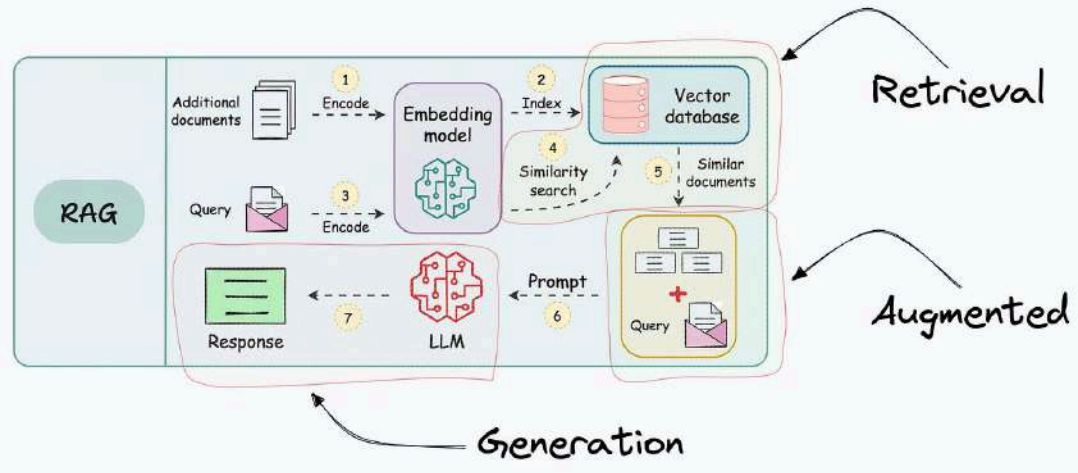
embeddings into the vector database. There's no need to repeat this again for the entire data)

Step 3: Use the same embedding model to embed the user query.

Step 4-5: Find the nearest neighbors in the vector database to the embedded query.

Step 6-7: Provide the original query and the retrieved documents (for more context) to the LLM to get a response.

In fact, even its name entirely justifies what we do with this technique:



Retrieval: Accessing and retrieving information from a knowledge source, such as a database or memory.

Augmented: Enhancing or enriching something, in this case, the text generation process, with additional information or context.

Generation: The process of creating or producing something, in this context, generating text or language.

Of course, there are many problems with RAG too, such as:

RAGs involve similarity matching between the query vector and the vectors of the additional documents. However, questions are structurally very different from answers.

Typical RAG systems are well-suited only for lookup-based question-answering systems. For instance, we cannot build a RAG pipeline to summarize the additional data. The LLM never gets info about all the documents in its prompt because the similarity matching step only retrieves top matches.

So, it's pretty clear that RAG has both pros and cons.

- We never have to fine-tune the model, which saves a lot of computing power.
- But this also limits the applicability to specific types of systems.

RAG vs REFRAG

Most of what we retrieve in RAG setups never actually helps the LLM.

As discussed earlier, in classic RAG, when a query arrives:

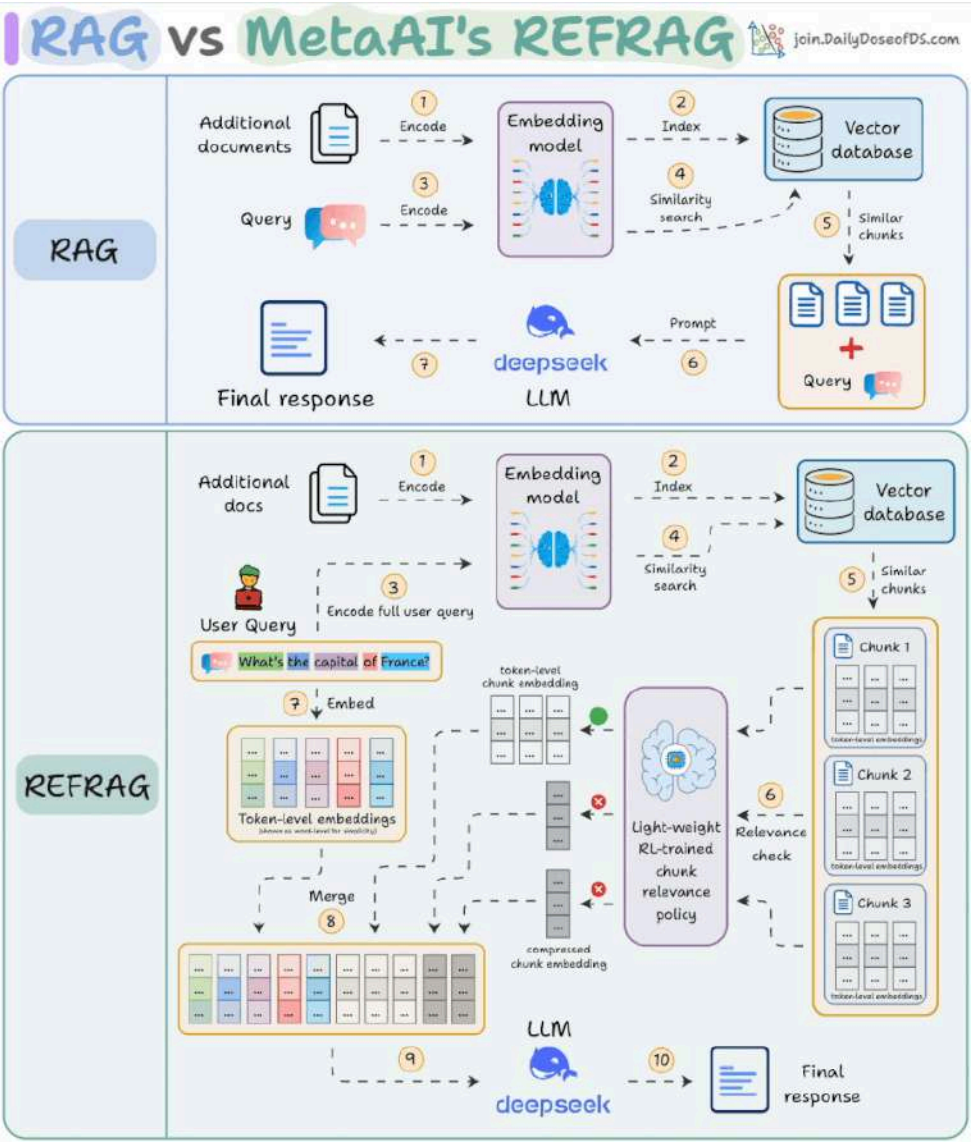
- You encode it into a vector.
- Fetch similar chunks from vector DB.
- Dump the retrieved context into the LLM.

It typically works, but at a huge cost:

- Most chunks contain irrelevant text.
- The LLM has to process far more tokens.
- You pay for compute, latency, and context.

That's the exact problem Meta AI's new method REFRAG solves.

It fundamentally rethinks retrieval and the diagram below explains how it works.



Essentially, instead of feeding the LLM every chunk and every token, REFRAG compresses and filters context at a vector level:

- Chunk compression: Each chunk is encoded into a single compressed embedding, rather than hundreds of token embeddings.
- Relevance policy: A lightweight RL-trained policy evaluates the compressed embeddings and keeps only the most relevant chunks.
- Selective expansion: Only the chunks chosen by the RL policy are expanded back into their full embeddings and passed to the LLM.

This way, the model processes just what matters and ignores the rest.

Here's the step-by-step walkthrough:

Step 1-2) Encode the docs and store them in a vector database.

Step 3-5) Encode the full user query and find relevant chunks. Also, compute the token-level embeddings for both the query (step 7) and matching chunks.

Step 6) Use a relevance policy (trained via RL) to select chunks to keep.

Step 8) Concatenate the token-level representations of the input query with the token-level embedding of selected chunks and a compressed single-vector representation of the rejected chunks.

Step 9-10) Send all that to the LLM.

The RL step makes REFRAG a more relevance-aware RAG pipeline.

Based on the research paper, this approach:

- has 30.85x faster time-to-first-token (3.75x better than previous SOTA)
- provides 16x larger context windows
- outperforms LLaMA on 16 RAG benchmarks while using 2–4x fewer decoder tokens.
- leads to no accuracy loss across RAG, summarization, and multi-turn conversation tasks

That means you can process 16x more context at 30x the speed, with the same accuracy.

RAG vs CAG

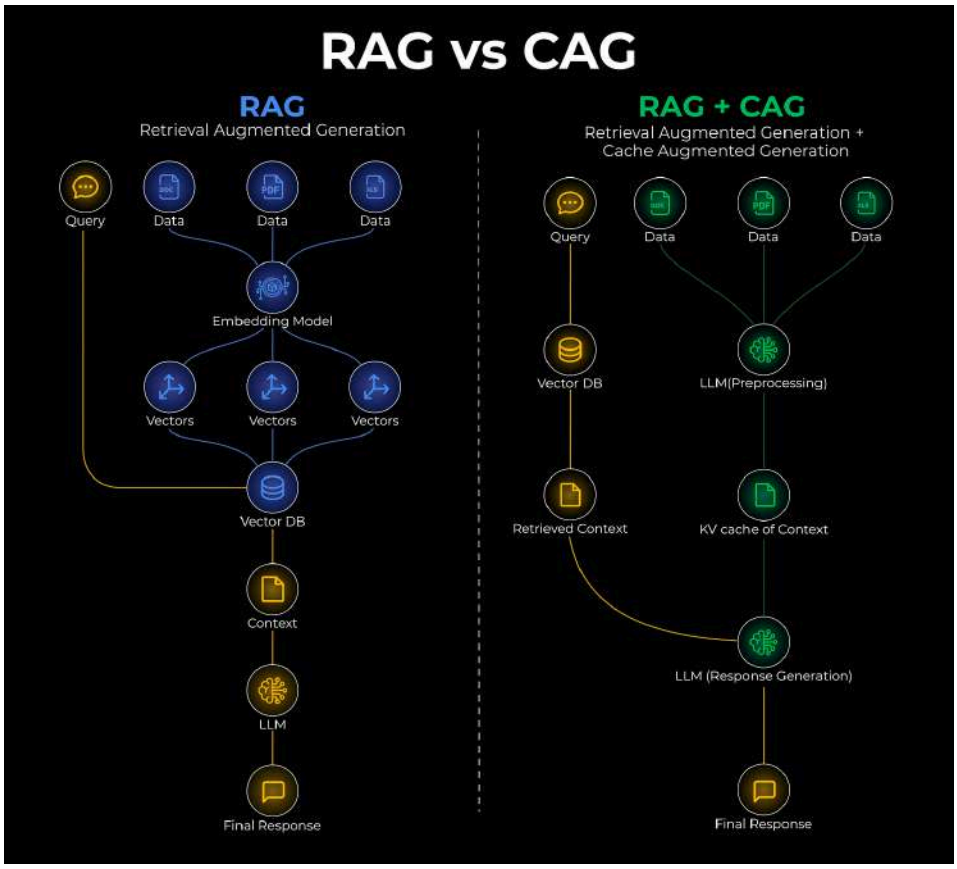
RAG changed how we build knowledge-grounded systems, but it still has a weakness.

Every time a query comes in, the model often re-fetches the same context from the vector DB, which can be expensive, redundant, and slow.

Cache-Augmented Generation (CAG) fixes this.

It lets the model "remember" stable information by caching it directly in the model's key-value memory.

And you can take this one step ahead by fusing RAG and CAG as depicted below:



Here's how it works in simple terms:

- In a regular RAG setup, your query goes to the vector database, retrieves relevant chunks, and feeds them to the LLM.
- But in RAG + CAG, you divide your knowledge into two layers.
    - The static, rarely changing data, like company policies or reference guides, gets cached once inside the model's KV memory.

- The dynamic, frequently updated data, like recent customer interactions or live documents, continues to be fetched via retrieval.

This way, the model doesn't have to reprocess the same static information every time.

It uses it instantly from cache, and supplements it with whatever's new via retrieval to give faster inference.

The key here is to be selective about what you cache.

You should only include stable, high-value knowledge that doesn't change often.

If you cache everything, you'll hit context limits, so separating "cold" (cacheable) and "hot" (retrievable) data is what keeps this system reliable.

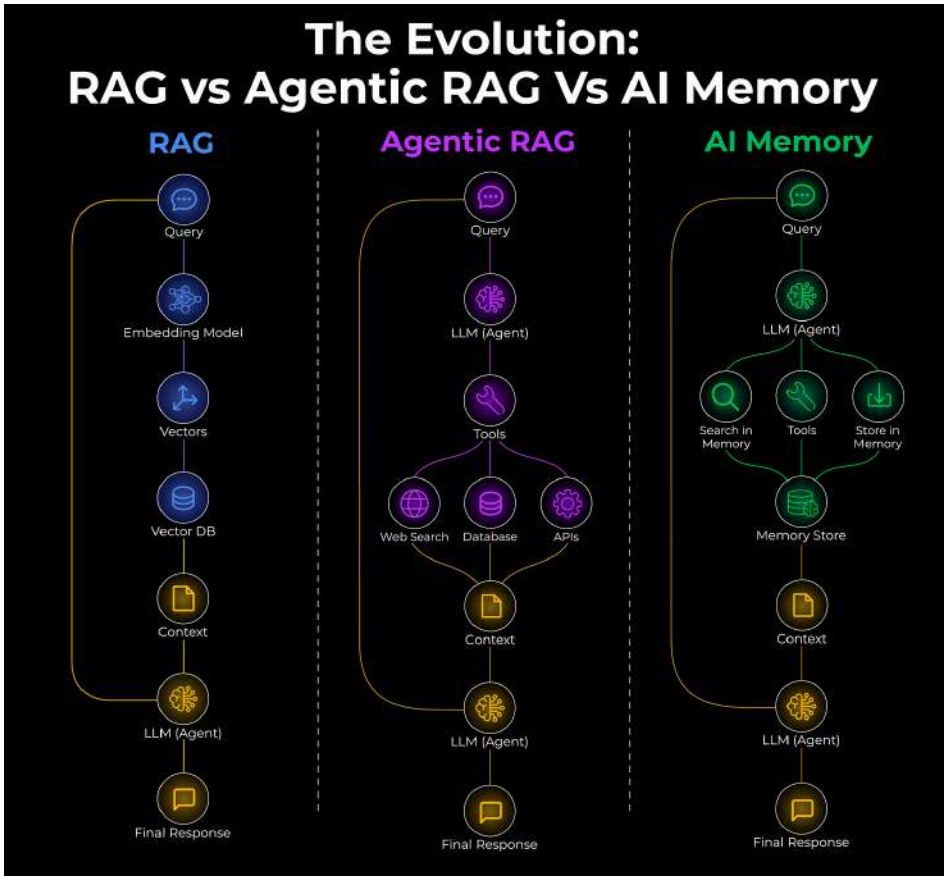
You can also see this in practice above.

Many APIs like OpenAI and Anthropic already support prompt caching, so you can start experimenting right away.

RAG, Agentic RAG and AI Memory

RAG was never the end goal. Memory in AI agents is where everything is heading.

Let's break down this evolution in the simplest way possible.



RAG (2020-2023):

- Retrieve info once, generate response
- No decision-making, just fetch and answer
- Problem: Often retrieves irrelevant context

Agentic RAG:

- Agent decides *if* retrieval is needed
- Agent picks *which* source to query
- Agent validates *if* results are useful
- Problem: Still read-only, can't learn from interactions

AI Memory:

- Reads AND writes to external knowledge
- Learns from past conversations

- Remembers user preferences, past context
- Enables true personalization

The mental model is simple:

- RAG: read-only, one-shot
- Agentic RAG: read-only via tool calls
- Agent Memory: read-write via tool calls

Here's what makes agent memory powerful:

The agent can now "remember" things, like user preferences, past conversations and important dates. All stored and retrievable for future interactions.

This unlocks something bigger: continual learning.

Instead of being frozen at training time, agents can now accumulate knowledge from every interaction. They improve over time without retraining.

Memory is the bridge between static models and truly adaptive AI systems.